

Data Structure 2

Paul Blondel

CNAM

September 1st, 2018

'I will, in fact, claim that the difference between a bad programmer and a good one is whether he considers his code or his data structures more important. Bad programmers worry about the code. Good programmers worry about data structures and their relationships.'

Linus Torvalds

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

What is a data structure?

A data structure is a **set of variables** of the **same nature**

A variable of a data structure can be accessed ...

- ... either directly by **giving a specific index** (a number)
- ... or by **traversing the structure in order**
- ... sometimes by **both methods** (directly and by traversing)

There are three types of sequential data structures:

① **contiguous** data structure

- ▶ Arrays
- ▶ Sequential files

② **linked** data structure

- ▶ linked-list

③ **abstract** data structure

- ▶ Stacks
- ▶ Queues

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

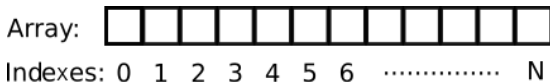
Contiguous?

Arrays are said contiguous because all their **variables are contiguously stored in memory** (= variables stored one after the other in memory).

Contiguous?

Arrays are said contiguous because all their **variables are contiguously stored in memory** (= variables stored one after the other in memory).

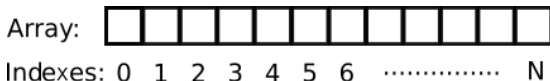
For instance, let's draw an empty array of size N :



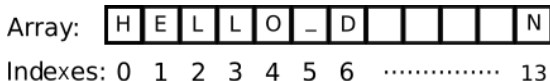
Contiguous?

Array are said contiguous because all their **variables are contiguously stored in memory** (= variables stored one after the other in memory).

For instance, let's draw an empty array of size N :



Let's now draw an array of characters containing "HELLO DONGGUAN":



To access a specific variable just specify the index, example: `Array[3] == L`, to access the next variable I just increase the index: `Array[4] == O`, and so on.

Examples of two basic array usage in pseudo-code:

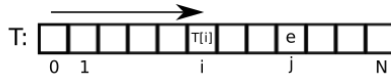
```
i = 0;  
while  $i < N$  do  
    | AnyFunction(Array[i]);  
    | i=i+1;  
end
```

Algorithm 1: Traversing an array, index after index...

```
i = 0;  
Sum = 0;  
while  $i < N$  do  
    | Sum = Sum + Array[i];  
    | i=i+1;  
end
```

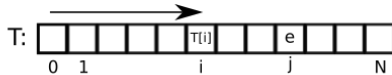
Algorithm 2: Sum of array elements

First, let's assume our array contains a list of **non-sorted elements**¹.



¹sorted list of elements: 1, 4, 7, 20, 55, non-sorted list of elements: 7, 5, 1, 2, 10

First, let's assume our array contains a list of **non-sorted elements**¹.

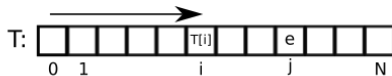


To search and find the "e" value in T we can perform a **sequential search**:

```
i = 0;  
while  $i \leq N$  and  $T[i] \neq e$  do  
  |  $i = i + 1$ ;  
end
```

¹sorted list of elements: 1, 4, 7, 20, 55, non-sorted list of elements: 7, 5, 1, 2, 10

First, let's assume our array contains a list of **non-sorted elements**¹.



To search and find the "e" value in T we can perform a **sequential search**:

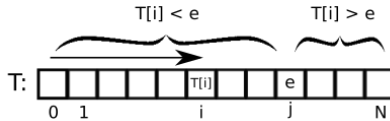
```
i = 0;  
while i <= N and T[i] != e do  
    | i=i+1;  
end
```

Here we have the following complexities:

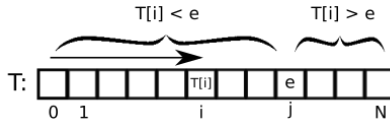
- at best: 1
- at worst: N
- in average: $\frac{N}{2}$ (if e belongs to T !)

¹sorted list of elements: 1, 4, 7, 20, 55, non-sorted list of elements: 7, 5, 1, 2, 10

First, let's now assume our array contains a list of **sorted elements**.



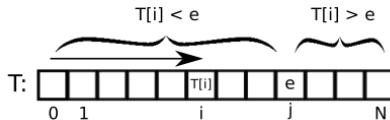
First, let's now assume our array contains a list of **sorted elements**.



Now that the array is sorted, the **sequential search** is slightly different:

```
i = 0;  
while i <= N and T[i] < e do  
    i=i+1;  
end
```


First, let's now assume our array contains a list of **sorted elements**.



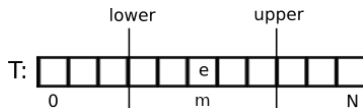
Now that the array is sorted, the **sequential search** is slightly different:

```
i = 0;  
while i <= N and T[i] < e do  
    | i=i+1;  
end
```

This time we have the following complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$ (if e belongs *or NOT* to T)

Let's now perform a **dichotomic search** on a sorted array:



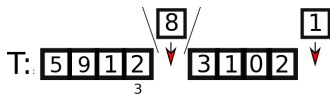
The *upper* and *lower* boundaries are adjusted step by step until finding "e":

```
lower = 0; upper = N;
found=False; m=0;
while lower<=upper and found
== False do
    m=(lower+upper) / 2;
    if T[m]==e then
        | found=True
    else
        | if T[m]<e then
        | | lower=m+1;
        | else
        | | upper=m-1;
        | end
    end
end
end
```

Here we have these complexities:

- at best: 1
- at worst: $\log_2(N)$

How inserting new values to my array?

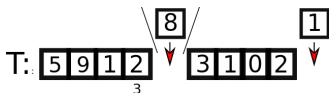


If your array is **NOT** sorted ...

- ... you can insert the element **at the end** (by default)
- or you can **specify after which index** you want to insert (ex: 3)

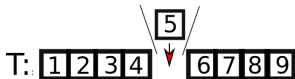
²Not doing this would make your array unsorted.

How inserting new values to my array?



If your array is **NOT** sorted ...

- ... you can insert the element **at the end** (by default)
- or you can **specify after which index** you want to insert (ex: 3)



If your array is **sorted** ...

- ... your insert should respect the **sorting order**!²

²Not doing this would make your array unsorted.

Let's assume inserting at the end of a **non-sorted** array is evident.

Now, for inserting an element in a specific position "pos", let's do:

```
reallocate_memory_and_copy(T);
```

```
i=pos;
```

```
while i < N do
```

```
    T[i+1] = T[i];
```

```
    i=i+1;
```

```
end
```

```
N=N+1;
```

```
T[pos]=e;
```

Here we have these complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

Let's assume inserting at the end of a **non-sorted** array is evident.

Now, for inserting an element in a specific position "pos", let's do:

```
reallocate_memory_and_copy(T);
```

```
i=pos;
```

```
while i < N do
```

```
    T[i+1] = T[i];
```

```
    i=i+1;
```

```
end
```

```
N=N+1;
```

```
T[pos]=e;
```

Here we have these complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

Note

- Inserting at the end is only $O(1)$, this should be preferred!
- The size of T has to be increased by 1 (for the new value)

Note

In the next slides, this "insert" function will call the pseudo-code above

Inserting a new value "e" in a **sorted array** requires a different approach:

```
reallocate_memory_and_copy(T);
```

```
// search part below
```

```
i=0;pos=0;
```

```
while  $i < N$  and  $T[i] < e$  do
```

```
    |  $i = i + 1$ ;
```

```
end
```

```
pos=i-1;
```

```
// insert part below
```

```
insert(e, pos);
```

Complexities for the search part:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

Note

The size of T has to be increased by 1 (for the new value)

Searching the insert place of a sorted array is better with a **binary search**:

```
// Search part below
lower=1; upper=N; pos=0; m=0;
if  $T[N] \leq e$  then
    | pos=pos+1;
else
    | while lower < upper do
        | m = (lower + upper) / 2;
        | if  $T[m] \leq e$  then
            | lower=m+1;
        | else
            | upper=m;
        | end
    | end
    pos=upper;
end

//Insert part below
insert(e, pos);
```

Complexities for the search part:

- at best: 1
- at worst: $\log_2(N)$

Removing an element (for both sorted and non-sorted arrays) requires:

- 1 to search for the position of the element
- 2 to "shrink" the array from that position

```
i=pos;  
while  $i < N$  do  
|    $T[i-1] = T[i];$   
|    $i=i+1;$   
end  
delete  $T[N-1];$   
 $N=N-1;$ 
```

Note

As the array "shrank" the previous last memory position must be deleted

Arrays...

- ...allow **direct access** to elements (just by giving the index)
- ...allow efficient element searches (**if sorted**)
- ...allow efficient element insertions (**if *not* sorted**)

Arrays...

- ...allow **direct access** to elements (just by giving the index)
- ...allow efficient element searches (**if sorted**)
- ...allow efficient element insertions (**if *not* sorted**)

Things to keep in mind

- The size of an array is fixed at the beginning: **beware of overflowing!**
- When needed, **changing the memory size of an array is slow!**

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files**
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

A sequential file is **finite** and contains **same type elements**, examples:

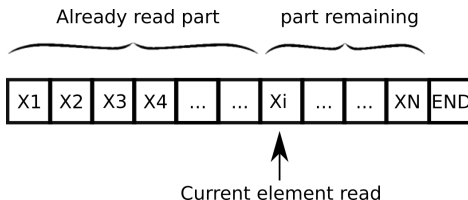
- file of the books of the library
- sound recording on a microphone
- ...

A sequential file is **finite** and contains **same type elements**, examples:

- file of the books of the library
- sound recording on a microphone
- ...

Its properties are:

- The structure is **fully ordered**
- The file only contains **elements of the same set**
- The **end of the file can be detected** (we don't need the size)



Several functions exist to handle files³:

- End of file, *eof(file)*:
 - ▶ returns *True* if the current read element is end of file.
- Access first element, *open(file)*:
 - ▶ current read element becomes first element.
- Read current element, *read(file, val)*:
 - ▶ copy current element in *val* and go to next element
- Create void file, *create(file)*:
 - ▶ create a void content file
- Write element, *write(file, val)*:
 - ▶ copy *val* at the end of the file
 - ▶ current read element is now last element

³The exact naming of the functions may be different depending on the programming language.

In order to find the element *val* in the sequential file, let's do:

```
found=False; c=void;
open(file);
while c != val and eof(file) != True do
    | read(file, c);
end
if eof(file) != True then
    | found=True;
end
```

Complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

In order to find the element *val* in the sequential file, let's do:

```
found=False; c=void;
open(file);
while c != val and eof(file) != True do
    | read(file, c);
end
if eof(file) != True then
    | found=True;
end
```

Complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

Notes

- *N* is the number of elements in the file
- we *cannot* use **binary search** optimization with sequential files

To insert an element "e" at the position k of a sequential file, let's do:

```
i=0;possible=False;
open(file);
create(file2);
while eof(file) != True and i<k
  do
    read(file, c);
    write(file2, c);
    i=i+1;
  end
if i==k then
  write(file2, e);
  while eof(file) != True do
    read(file,c);
    write(file2,c);
  end
  possible=True;
else
  possible=False;
end
```

Complexities:

- at best: N
- at worst: N
- in average: N

To remove the element at the position k of a sequential file, let's do:

```
i=0;possible=False;
open(file);
create(file2);
while eof(file) != True do
  if i!=k then
    read(file, c);
    write(file2, c);
  else
    read(file, c);
  end
  i=i+1;
end
if i<k then
  possible=False;
else
  possible=True;
end
```

Complexities:

- at best: N
- at worst: N
- in average: N

Sequential files...

- ...have a **sequential access** (calling "*read(file, c)*" to "*eof(file)*")
- ...have to be **rebuilt in case of updates** (insertions, or deletions)
- ...have **implemented functions** to access elements
- ...can be **read from mass-storage memory**

Sequential files...

- ...have a **sequential access** (calling "*read(file, c)*" to "*eof(file)*")
- ...have to be **rebuilt in case of updates** (insertions, or deletions)
- ...have **implemented functions** to access elements
- ...can be **read from mass-storage memory**

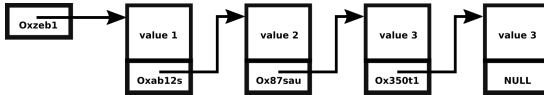
Things to keep in mind

- **Not optimized for fast accesses and updates**
- More a **long term data storing structure** (in mass-storage memory)
- **Algorithm costs are proportional to the size of the file**

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list**
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

Linked?

Because each of its element contains the address to go to the next element⁴



Properties:

- A linked-list is **NOT** contiguous in memory
- The memory for each **new** element is dynamically allocated
- Like arrays, we can allocate N elements when creating the list

⁴Note that the first "element" only contains an address, the last element contains the NULL address.

In order to search an element "e" in a linked-list, let's do⁵:

```
p=List.address;  
while p != NULL and p.value != e  
do  
  | p = p.address;  
end
```

Complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

⁵Accessing the address of an element p : $p.address$, and accessing the value: $p.value$.

In order to search an element "e" in a linked-list, let's do⁵:

```
p=List.address;  
while p != NULL and p.value != e  
do  
  | p = p.address;  
end
```

Complexities:

- at best: 1
- at worst: N
- in average: $\frac{N}{2}$

Notes

- Traversing the linked-list is just fetching the next address until the NULL pointer
- N is the number of elements in the linked-list

⁵Accessing the address of an element p : $p.address$, and accessing the value: $p.value$.

The easiest way to insert a new element is **at the end** of the linked-list:

```
create(new_element);
```

```
//Search part below
```

```
p=List.address;
```

```
while  $p \neq \text{NULL}$  do
```

```
    |   p = p.address;
```

```
end
```

```
//Insert part below
```

```
p.address=get_address(new_element);
```

- Complexity search part: N
- Complexity insert part: 1

You can also insert a new element **in the linked-list** (ex: after k elements):

```
create(new_element);

//Search part below
i=0;
p=List.address;
while  $p \neq \text{NULL}$  and  $i < k$  do
    |   p = p.address;
    |   i=i+1;
end

//Insert part below
new_element.address=p.address;
p.address=get_address(new_element);
```

- Complexity search part: $O(N)$
- Complexity insert part: 1

You can also insert a new element **in the linked-list** (ex: after k elements):

```
create(new_element);

//Search part below
i=0;
p=List.address;
while  $p \neq \text{NULL}$  and  $i < k$  do
    |   p = p.address;
    |   i=i+1;
end

//Insert part below
new_element.address=p.address;
p.address=get_address(new_element);
```

- Complexity search part: $O(N)$
- Complexity insert part: 1

Note

If you know the address after which you want to add an element:
complexity = $O(1)$

Removing an element with value e is **swapping** addresses:

```
old_address=NULL;

//Search part below
p=List.address;
if List.address.value != e then
    | while p.address != NULL and
    |   p.address.value != e do
    |   | p=p.address;
    | end
end

//Remove part below
if p != NULL then
    | old_address = p;
    | if p.address != NULL then
    |   | p = p.address.address;
    | else
    |   | p = NULL;
    | end
    | delete(old_address);
end
```

- Complexity search part: $O(N)$
- Complexity remove part: 1

Linked-lists...

- ...offer **more memory flexibility**
- ...allow **fast element insertions**:
 - ▶ no need to allocate $N + 1$ memory cells
 - ▶ no need to copy N previous cells
- ...allow **fast element deletions** (same as above)
- ...can **only be accessed sequentially** (following the addresses ...)
- ...are **slower to traverse than arrays** (memory is not contiguous)

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks**
- 6 Queues
- 7 Hash-table and hash function

Abstract?

A stack is said *abstract* because:

- It is **defined by its behavior**, not its memory management
- It can be based on *arrays* or *linked-list*

Properties:

- A new inserted element takes the first position⁶
- Reading the stack = reading the first position⁶
- The read element is also removed from the stack⁶...

⁶This is the *LIFO* principle: Last In First Out.

Array-based stack:

```
level=0;
allocate_memory(Array, N);

// Insert stack method
Function push(value) is
|   if level < N then
|   |   level = level+1;
|   |   Array[level] = value;
|   end
end

// Read stack method
Function pop() is
|   if level > 0 then
|   |   level=level-1;
|   |   return Array[level];
|   end
end
```

Linked-list based stack:

```
create_list(List);

// Insert stack method
Function push(value) is
|   create_element(element, value);
|   old_address=List.address;
|   List.address=element;
|   element.address=old_address;
end

// Read stack method
Function pop() is
|   if List.address != NULL then
|   |   p = List.address;
|   |   val = p.value;
|   |   delete(p);
|   |   return val;
|   end
end
```

Common function naming

push: inserts element in stack, *pop*: read the stack.

Stacks...

- ...help organize data in terms of insertions and deletions:
 - ▶ it **conserves the order of insertion**
 - ▶ you can **easily reverse back**
- ...automatically **remove the last read element**

Example of usage

The undo mechanism (CTR+Z) in text editors uses a stack.

- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues**
- 7 Hash-table and hash function

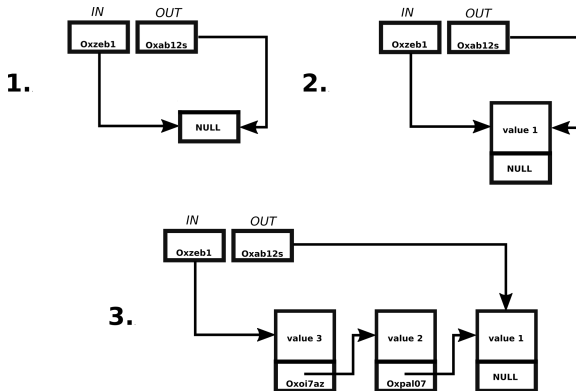
What's different this time?

Queues are also *abstract* BUT are based on the *FIFO*⁷ principle

Properties:

- A new inserted element takes the first position⁶
- Reading the queue = reading the *last* position⁶
- The read element is also removed from the queue⁶...

⁷FIFO: First In First Out (see that as a sort of *waiting line*)



- ❶ Initial state: both *in* and *out* addresses point to NULL⁸
- ❷ Inserting element: *in* points to new first element⁸
- ❸ Inserting elements: *in* points to new first element, *out* points to last⁸

⁸In this example, we illustrated a linked-list based queue.

Array-based queue:

```
number=0;first=0;last=0;
allocate _memory(Array, N);

// Insert queue method
Function enqueue(value) is
    if number < N then
        number = number+1;
        Array[last] = value;
        last = (last+1) % N;
    end
end

// Read queue method
Function dequeue() is
    if number > 0 then
        number=number-1;
        r = Array[first];
        first = (first+1) % N;
        return r;
    end
end
```

Linked-list based queue:

```
In=NULL;Out=NULL;

// Insert queue method
Function enqueue(value) is
    create _element(element, value);
    if Out == NULL then
        Out = element;
    else
        element.address = In;
    end
    In = element;
end

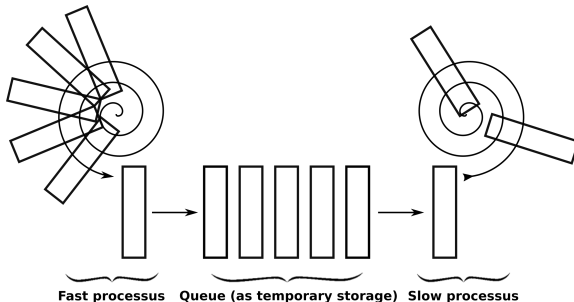
// Read queue method
Function dequeue() is
    if In != NULL then
        p = In;
        In = In.address;
        val = p.value;
        delete(p);
        return val;
    end
end
```

Common function naming

enqueue: inserts element in queue, *dequeue*: read the queue.

Queues...

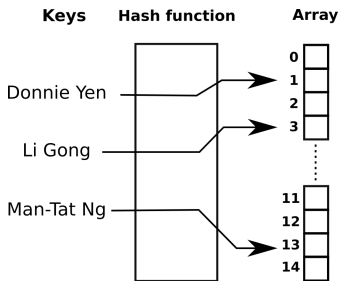
- ...help organize data in terms of insertions:
 - ▶ first element *inserted* become first element *out* (**FIFO**)
 - ▶ see that as a *waiting list*
- ...automatically **remove the last read element**
- ...are **useful for asynchronous data distribution** (buffering):
 - ▶ newly added data is stored in the queue (as temporary storage)
 - ▶ when needed, data is extracted from the queue



- 1 Sequential data structures
- 2 Contiguous data structure - Arrays
- 3 Contiguous data structure - Sequential files
- 4 Linked data structure - Linked-list
- 5 Abstract data-structure - Stacks
- 6 Queues
- 7 Hash-table and hash function

Hash function?

A hash function takes a **key in input** and **transforms it into an index**

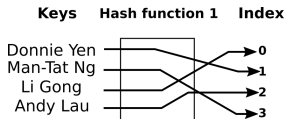


Properties:

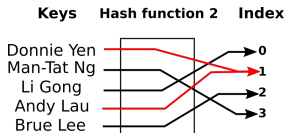
- An hash function is an essential part of a hash-based data structure
- Hash-based data structures⁶ allow direct data access (through *keys*)
- Keys can be of various nature (words, sentences, numbers, and so on)

⁶Also known as *hash table* (or *hash map*)

The ideal *hash function* should be *bijective*⁹:



In reality most *hash functions* are not *bijective*, there are **collisions**:



What criteria for a good hash function?

- Have the **fewest possible collisions**
- **Uniformly** distribute the indexes (spreading the data along the array)

⁹bijective = for each input *one and only one* output associated

A good example of hash function is the h_1 function:

$$h_1(\text{key}) = (f(\text{key}[0]) + f(\text{key}[1]) \times 26 + f(\text{key}[2]) \times 26^2 + \dots + f(\text{key}[k]) \times 26^k) \% N$$

Where the f function gives the alphabet number, ex: $f(c) == 2$.

Example (let's say $N = 100000$):

$$h_1(\text{hello}) = (f(h) + f(e) \times 26 + f(l) \times 26^2 + f(l) \times 26^3 + f(o) \times 26^4) \% N$$

$$h_1(\text{hello}) = (7 + 4 * 26 + 11 * 26^2 + 11 * 26^3 + 14 * 26^4) \% 100000$$

$$h_1(\text{hello}) = 98547$$

Note

N is the maximum number of elements that can be stored in the hash-table

Although quite rare, collisions can be managed using:

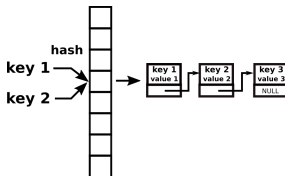
- *external chaining*

- ▶ interesting if we **don't know how many keys we have to handle**
- ▶ interesting if we have **lots of insertions/deletions**
- ▶ but, likely to consume a **lot more memory**

- or *internal chaining*

- ▶ interesting if we **KNOW how many keys we have to handle ($< N$)**
- ▶ but, if **many collisions to handle = slower** than external chaining
- ▶ is a **lighter structure** (consuming less memory)

External chaining:



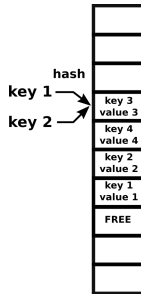
- Insertion:

- ▶ if void, we **insert the first linked-list element**
- ▶ if no void (collision occurred), we **add a linked-list element**

- Reading:

- ▶ we **sequentially search** in the linked-list
- ▶ finding an element **with same key**: we found the right element

Internal chaining:



- Insertion:
 - ▶ if void, we **insert the element**
 - ▶ if it collides, we **search for next FREE element and insert**
- Reading:
 - ▶ if we have the **same key: we take the value**
 - ▶ if not, we **search in the next elements**

Complexities for accessing a hash-table element (insert, read, delete):

Complexities	At best	At worst
No collision management	1	1
External chaining	1	N
Internal chaining	1	N

Hash-tables...

- ...are **fast in insertions, reading and deletions**:
 - ▶ **no need to search** in the data structure (if the key is well thought)
 - ▶ if **collisions are rares: complexity near 1**
- ...**cannot be accessed sequentially**
- ...are **difficult to design** (better use default ones):
 - ▶ **bad hash function = more collisions = slow**
 - ▶ **not enough memory (N small) = more collisions = slow**

Example

Hash-table giving the international telephone code knowing the country:
Hash[china] == 86, Hash[france] == 33, hash[germany] == 49